HW7: Multi-View Point Cloud Grasping on the Kinova Gen3 Lite

Anshul Raje, Eda Guven, Harsh Shah, Shreehan S. Kate
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: {arraje, egven, hshah2, skate}@wpi.edu

I. Objective and System Overview

The objective of this assignment was to transfer the grasping pipeline developed in HW 6 from simulation to a real robot. The simulation setup used two fixed RGB-D cameras, whereas the real-robot setup uses a wrist-mounted RGB-D camera. Because the camera moves with the robot, observations from different viewpoints must be transformed into a common reference frame before they can be combined.

Our implementation captures point clouds from three observation poses, transforms them into the robot base frame, and merges them into an augmented multi-view cloud. The cloud is then processed through workspace filtering, table removal, object clustering, target selection, surface-normal estimation, and grasp synthesis. Finally, the robot moves to a pre-grasp pose, approaches the object, closes the gripper, and lifts.

The implementation is modular and parameterized. Observation poses, workspace bounds, filtering settings, target selection, grasp constraints, and motion settings can be changed without rewriting the overall pipeline.

II. Calibration and Initialization

A. Eye-in-Hand Calibration

The implementation includes an optional ArUco-based eye-in-hand calibration procedure, launched using:

```
ros2 run kinova_grasp calibrate
```

The procedure estimates the rigid transform between `end_effector_link` and `camera_link`, and stores it in `views.yaml`. An existing calibration can be reused if available.

Let B, E, and C denote the robot base, end-effector, and camera frames, respectively. Using the convention that ${}^{A}T_{D}$ maps coordinates from frame D into frame A, the camera-to-base transform at acquisition time t_i is

$$ {}^{B}T_{C}(t_i) = {}^{B}T_{E}(t_i)\,{}^{E}T_{C}. \tag{1} $$

A point expressed in homogeneous camera coordinates is transformed as

$$ {}^{B}\tilde{\mathbf{p}} = {}^{B}T_{C}(t_i)\,{}^{C}\tilde{\mathbf{p}}. \tag{2} $$

If the point cloud is published in a camera optical frame rather than `camera_link`, the corresponding optical-frame transform must also be included through TF. The transform should correspond to the point-cloud acquisition time, since the camera pose relative to the base changes with wrist motion.



Fig. 1. Robot at the home configuration, with the wrist camera facing down toward the workspace.

B. Home Configuration

Before point-cloud acquisition, the robot moves to predefined home joint angles. This provides a repeatable starting configuration for observation motions and subsequent grasp execution. The home configuration is shown in Fig. 1.

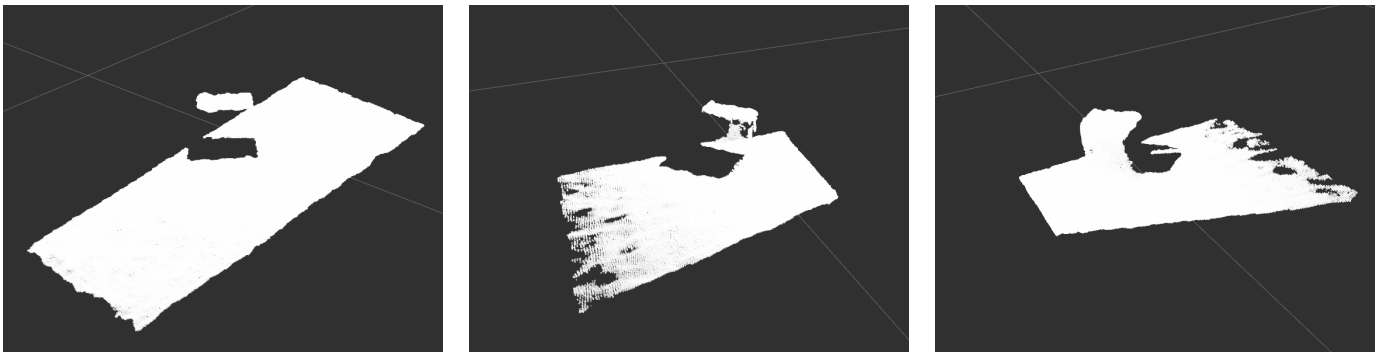
III. Multi-View Point-Cloud Acquisition

The pipeline uses three meaningfully different viewpoints:

1) Top view directed approximately straight down.
2) $+y$ side view tilted by approximately 40°.
3) $-y$ side view tilted by approximately 40°.

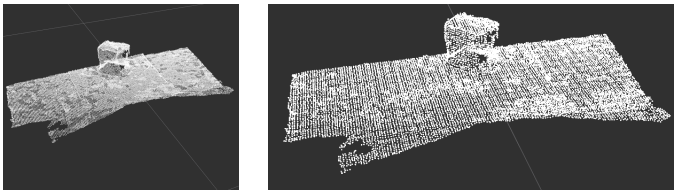
The side views provide observations of surfaces that may be occluded in the top view. Between the two side views, the robot returns through the top-view configuration rather than moving directly across the object region. This routing is intended to reduce interference with the objects, although the full robot and camera geometry must still be checked for clearance.

Five point-cloud frames are acquired at each viewpoint after the robot reaches the observation pose. Points outside a 0.12–$0.55\,\mathrm{m}$ depth range are discarded, which also removes the gripper fingers from the view. Every cloud is transformed into `base_link` and coarsely cropped to the workspace before being saved or combined with other observations. The



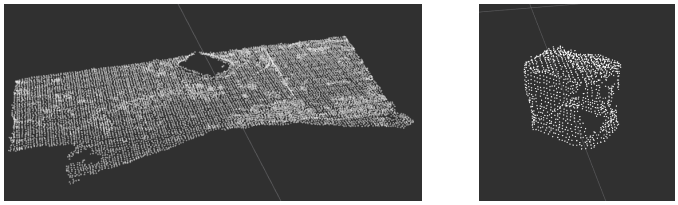
(a) Top view (b) $+y$ side view (c) $-y$ side view

Fig. 2. Point clouds acquired at each of the three observation viewpoints, shown individually after transformation into `base_link`. Each view captures surfaces that are partially occluded in the others.



(a) Merged cloud (b) Cropped and downsampled

Fig. 3. Multi-view cloud (a) after fusing the three viewpoints and (b) after workspace cropping and $5\,\mathrm{mm}$ voxel downsampling.



(a) Table-plane inliers (b) Target cluster

Fig. 4. Segmentation of the filtered cloud: (a) points assigned to the table by RANSAC, leaving a gap where the object stands, and (b) the remaining object points after Euclidean clustering.

observation poses are configurable to accommodate different workspace layouts. The individual clouds from each viewpoint are shown in Fig. 2.

IV. Cloud Fusion and Workspace Filtering

A. Multi-View Fusion

After transformation into the base frame, the observations are merged:

$$P_{\mathrm{merged}} = \bigcup_i \left\{ {}^{B}T_{C}(t_i)\tilde{\mathbf{p}} \;:\; \mathbf{p} \in P_i \right\}. \tag{3}$$

The TF-based transforms provide the initial alignment. Each side view is then refined against the top view using point-to-plane iterative closest point (ICP) registration. Because ICP can converge to a wrong alignment, its correction is accepted only if it is small (at most $20\,\mathrm{mm}$ and 5°) and the resulting overlap with the top view is at least 30%; otherwise, the TF-only alignment is kept. Alignment quality therefore still depends on hand-eye calibration accuracy and robot pose accuracy.

Outlier removal and voxel downsampling reduce noise and redundant samples in the merged cloud. The resulting augmented cloud combines the complementary observations from the three viewpoints, as shown in Fig. 3.

B. Workspace Cropping and Downsampling

The merged cloud is cropped to a configurable workspace box in `base_link`. This removes background points and geometry outside the intended grasping region. The cropped cloud is then voxel-downsampled using a voxel size of $5\,\mathrm{mm}$.

The workspace bounds used in the experiment were:

$$x \in [0.30,\ 0.75]\ \mathrm{m}, \tag{4}$$

$$y \in [-0.40,\ 0.40]\ \mathrm{m}, \tag{5}$$

$$z \in [-0.06,\ 0.30]\ \mathrm{m}. \tag{6}$$

The lower z bound lies just below the table top ($z \approx -0.043\,\mathrm{m}$), so the table remains in the cloud for plane estimation. The resulting cloud is shown in Fig. 3b.

V. Table Removal and Object Segmentation

A. RANSAC Table-Plane Estimation

A RANSAC plane fit identifies the table surface. The estimated plane is represented by

$$\mathbf{n}_t^T \mathbf{p} + d = 0, \tag{7}$$

where $\mathbf{n}_t$ is the unit table normal, oriented upward.

Table inliers are removed from the cloud. The remaining points are used for object clustering, while the estimated plane is retained for evaluating contact height and closing-axis inclination. The table inliers are shown in Fig. 4a.

B. Euclidean Clustering and Target Selection

Euclidean clustering separates the remaining points into object candidates based on spatial proximity. Ideally, each object

produces one cluster. However, touching objects or unsuitable clustering thresholds can cause clusters to merge or split. The test scene contained a single object, so clustering produced one cluster, which was selected as the target (Fig. 4b).

VI. Surface Normals and Grasp Synthesis

A. Surface-Normal Estimation

Surface normals are estimated for the target cluster and oriented toward the object interior. These normals are used to identify approximately opposing contact surfaces.

B. Candidate Contact Pairs

Each candidate grasp contains two contact points, $\mathbf{p}_1$ and $\mathbf{p}_2$. The closing axis is

$$\hat{\mathbf{c}} = \frac{\mathbf{p}_2 - \mathbf{p}_1}{\|\mathbf{p}_2 - \mathbf{p}_1\|}. \tag{8}$$

Candidates must satisfy four constraints:

1) **Contact separation:**

$$0.01 \leq \|\mathbf{p}_2 - \mathbf{p}_1\| \leq 0.08 \quad \text{meters}. \tag{9}$$

2) **Normal alignment:** Each inward normal must lie within 25° of the direction toward the opposing contact:

$$\hat{\mathbf{n}}_1^T \hat{\mathbf{c}} \geq \cos(25^\circ), \qquad \hat{\mathbf{n}}_2^T (-\hat{\mathbf{c}}) \geq \cos(25^\circ). \tag{10}$$

3) **Closing-axis inclination:** The closing axis must lie within 30° of the table plane:

$$|\mathbf{n}_t^T \hat{\mathbf{c}}| \leq \sin(30^\circ). \tag{11}$$

4) **Contact clearance:** Both contacts must be at least $12\,\mathrm{mm}$ above the table:

$$\mathbf{n}_t^T \mathbf{p}_j + d \geq 0.012 \quad \text{meters}, \qquad j \in \{1, 2\}. \tag{12}$$

The inclination constraint rejects near-vertical closing directions, which could require a finger to pass underneath the object and into the table. The contact-height constraint provides additional fingertip clearance. These geometric checks reduce collision risk but do not replace collision checking of the complete gripper geometry.

Among valid candidates, the scoring procedure favors pairs whose contacts lie close to the target centroid, whose normals are most directly opposed, and whose closing direction runs across, rather than along, the robot-to-object line.

The grasp position is the contact midpoint:

$$\mathbf{p}_g = \frac{\mathbf{p}_1 + \mathbf{p}_2}{2}. \tag{13}$$

The selected contact pair and its normals are shown in Fig. 5.

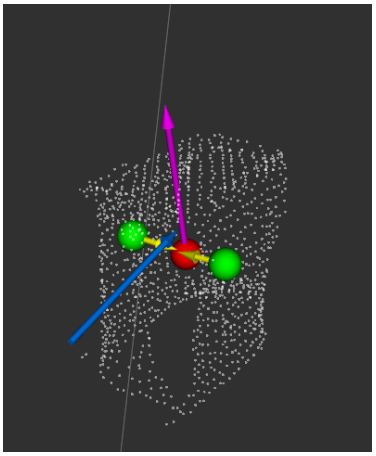


Fig. 5. Grasp synthesized on the target cluster: centroid (red), selected contact points (green), contact normals (yellow), PCA major axis (magenta), and approach vector ending at the grasp center (blue).

VII. Grasp Orientation and Pre-Grasp Pose

Principal component analysis (PCA) estimates the target cluster's major axis, denoted by $\hat{\mathbf{m}}$. The approach axis is computed from the cross product of the major axis and closing axis:

$$\hat{\mathbf{a}} = s\,\frac{\hat{\mathbf{m}} \times \hat{\mathbf{c}}}{\|\hat{\mathbf{m}} \times \hat{\mathbf{c}}\|}, \qquad s \in \{-1, +1\}. \tag{14}$$

The sign is chosen so that the approach points downward and, where possible, outward from the robot; the pre-grasp therefore lies above the object on the robot side. Here, $\hat{\mathbf{a}}$ denotes the direction of motion from pre-grasp toward grasp. The closing and approach axes define the grasp orientation according to the gripper's frame convention.

If the major axis and closing axis are nearly parallel, the cross product becomes ill-defined. In that case, the approach falls back to the preferred direction (outward from the robot and downward), projected to be perpendicular to the closing axis.

The pre-grasp is placed $10\,\mathrm{cm}$ behind the grasp:

$$\mathbf{p}_{\mathrm{pre}} = \mathbf{p}_g - 0.10\hat{\mathbf{a}}. \tag{15}$$

The resulting major axis and approach vector for the target object are shown in Fig. 5.

VIII. Robot Motion and Grasp Execution

A. Home-to-Pre-Grasp Motion

The `pregrasp_motion` parameter selects one of two methods:

- `planned`: motion planning using RRTConnect; this is the current configuration.
- `cartesian`: a straight-line Cartesian alternative.

The robot moves from home to the pre-grasp pose using the selected method. If this motion fails, the grasp attempt is marked as failed and does not continue to the approach stage.

B. Cartesian Approach

From pre-grasp, the robot attempts a straight-line approach of up to $10\,\mathrm{cm}$ along $\hat{\mathbf{a}}$.

If the table obstructs the path, the approach stops $1\,\mathrm{cm}$ before the first detected collision. The attempt fails if the executable approach distance is less than half of the requested distance.

C. Gripper Closure and Lift

After a successful approach, the gripper closes. The robot then lifts $10\,\mathrm{cm}$ vertically using a straight-line Cartesian motion from the actual approach stopping pose.

IX. Parameter Summary

Table I lists the main settings and values used in the experiments. Most of them are set in the YAML configuration files and can be changed without modifying the pipeline code.

TABLE I
MAIN CONFIGURABLE SETTINGS AND VALUES USED IN THE PIPELINE.

Setting	Value
Hand-eye calibration	ArUco marker, Tsai method; stored in `views.yaml`
Observation poses	Top, $+y$ side, $-y$ side
Side-view tilt	$\approx 40^\circ$
Frames per viewpoint	5
Depth range	0.12–0.55 m
Common frame	`base_link`
Outlier removal	Statistical, $k = 20$, 2σ
Registration	TF, refined by point-to-plane ICP
ICP acceptance	$\leq 20\,\mathrm{mm}$, $\leq 5^\circ$, overlap $\geq 30\%$
Workspace bounds	See Section IV-B
Voxel size	$5\,\mathrm{mm}$
RANSAC plane	$1\,\mathrm{cm}$ threshold, 200 iterations
Euclidean clustering	$2\,\mathrm{cm}$ tolerance, 100–25000 points
Target selection	`target_cluster`; ordered by centroid y
Normal search radius	$1\,\mathrm{cm}$
Contact separation	1–8 cm
Normal-alignment tolerance	25°
Closing-axis inclination	$\leq 30^\circ$ from table plane
Minimum contact height	$12\,\mathrm{mm}$ above table
Pre-grasp offset	$10\,\mathrm{cm}$
Pre-grasp motion	`planned` (RRTConnect)
Approach collision margin	$1\,\mathrm{cm}$
Minimum executable approach	50% of requested distance
Vertical lift	$10\,\mathrm{cm}$
Velocity/acceleration scaling	0.1

X. Experimental Results and Discussion

The pipeline was evaluated over 150+ grasp attempts, run across several team members' laptops. The test objects covered a range of shapes and materials: three cuboids of different sizes, a soup can, a Spam can, a large cylindrical coffee jar, a bleach bottle, a pear, a banana, and a soft Lego brick.

The main limitations are sensitivity to calibration and TF accuracy, incomplete geometry due to occlusion, and dependence on filtering and segmentation thresholds. ICP refinement corrects only small misalignments; larger calibration errors, or a rejected ICP correction, can still appear as duplicated or offset surfaces in the merged cloud. PCA-based orientation may also be ambiguous for approximately symmetric objects or unstable when the observed geometry is incomplete.



Fig. 6. Final lift of the target object on demo day.

In practice, the most common failure was the home-to-pre-grasp motion. Grasp synthesis selects the grasp from object geometry alone, without considering whether the arm can reach it, so many pre-grasp poses were infeasible within the Gen3 Lite's reach and joint limits. Straight-line Cartesian motion often failed near the wrist singularity, which is why planned (RRTConnect) motion was adopted, but even planned motion could not reach some poses. Checking reachability during grasp scoring, or falling back to the next-best candidate, would address this.

Fig. 6 shows the final lift of an object executed on demo day. Videos of our runs can be found here: *link*.